

GUÍA DE LABORATORIO N.º 4

Técnicas Estadísticas para el Análisis de Datos con Python

Descriptiva robusta, intervalos de confianza, pruebas de hipótesis y experimentos A/B

Semana / Tema	Semana 4: Uso de técnicas estadísticas para el análisis de datos
Modalidad	Individual (cada estudiante desarrolla y entrega su propio notebook)
Entorno	Google Colab o Jupyter local (Python 3.10 o superior)
Librerías	pandas, NumPy, SciPy (≥ 1.11), statsmodels (≥ 0.14), Matplotlib
Entregable	Notebook Lab04_Apellido_Codigo.ipynb y su exportación en PDF en el Aula Virtual

1. Objetivos del laboratorio

Al finalizar el laboratorio, el estudiante será capaz de:

1. Calcular e interpretar resúmenes descriptivos robustos por grupos y comparar criterios de detección de valores atípicos.
2. Evaluar la forma de una distribución con gráficos Q-Q y construir intervalos de confianza paramétricos y por bootstrap.
3. Seleccionar y ejecutar pruebas de comparación de grupos (Welch, Mann-Whitney, Kruskal-Wallis) con tamaño del efecto y corrección por comparaciones múltiples.
4. Evaluar asociaciones entre variables categóricas (chi-cuadrado, V de Cramér) y ordinales (Spearman), incluyendo el análisis por segmentos.
5. Analizar un experimento A/B a partir de datos crudos: verificación de SRM, prueba de proporciones, análisis por segmentos y efecto del peeking.
6. Aplicar de forma autónoma el flujo estadístico completo a un problema nuevo y comunicar conclusiones orientadas a la decisión.

2. Materiales y datos

Los tres archivos de datos se entregan junto con esta guía en el Aula Virtual. Son **datos sintéticos** generados con semilla fija, por lo que todos los resultados son reproducibles y coinciden con los valores esperados que muestra la guía.

Archivo	Registros	Uso en el laboratorio
pedidos_mercaandina_2026.csv	12 000 filas, 6 columnas	Partes 1 a 5 (laboratorio guiado)
experimento_checkout_2026.csv	40 250 filas, 7 columnas	Parte 6 (experimento A/B)
transacciones_api_billetera_2026.csv	18 000 filas, 9 columnas	Actividad propuesta (40 minutos)

Tabla 1. Archivos de datos del laboratorio

El diccionario de datos de cada archivo se encuentra en el **Anexo A**. Lea las descripciones de las variables antes de comenzar.

2.1 Distribución del tiempo

Parte	Contenido	Tiempo
0	Preparación del entorno	5 min
1	Exploración y descriptiva robusta	12 min
2	Valores atípicos	8 min
3	Forma de la distribución e intervalos de confianza	12 min

4	Comparación de grupos	15 min
5	Asociación entre variables	10 min
6	Experimento A/B desde datos crudos	18 min
7	Actividad propuesta individual	40 min
	Total	120 min

Tabla 2. Cronograma sugerido

Reglas de trabajo

- El laboratorio es **individual**. Puede consultar la teoría de la semana 4 y la documentación oficial de las librerías, pero el código y las interpretaciones deben ser propios.
- Cada paso debe quedar en su propia celda de código, precedida por una celda Markdown con el título del paso (por ejemplo, «Paso 2.1»).
- Las **preguntas de control** se responden en celdas Markdown inmediatamente después del resultado, con 2 a 4 oraciones y citando los valores obtenidos.
- Si usa un asistente de IA generativa, debe indicarlo en una celda final e incluir qué verificó por su cuenta.

Parte 0. Preparación del entorno

Paso 0.1. Crear el notebook

Opción A – Google Colab:

1. Ingrese a <https://colab.research.google.com> con su cuenta institucional @unmsm.edu.pe.
2. Seleccione Archivo → Nuevo notebook.
3. Cambie el nombre del notebook a Lab04_Apellido_Codigo.ipynb (por ejemplo, Lab04_Quispe_20200123.ipynb).
4. En la primera celda Markdown escriba: nombre completo, código de matrícula y fecha.

Opción B – Jupyter local (recomendada): cree un entorno virtual e instale las librerías con el siguiente comando en la terminal; luego ejecute jupyter lab en la carpeta donde guardó los archivos CSV.

```
python -m venv lab04
# Windows: lab04\Scripts\activate | Linux/macOS: source lab04/bin/activate
pip install pandas numpy scipy statsmodels matplotlib jupyterlab
```

Paso 0.2. Cargar los archivos de datos

En Colab, haga clic en el ícono de carpeta del panel izquierdo y arrastre los tres archivos CSV, o ejecute la siguiente celda y selecciónelos. Los archivos subidos a Colab se eliminan al cerrar la sesión: si se desconecta, vuelva a subirlos.

```
from google.colab import files
subidos = files.upload() # seleccione los 3 archivos CSV
!ls -lh *.csv
```

Paso 0.3. Importar librerías y verificar versiones

Propósito: dejar preparado el entorno con todas las funciones que se usarán y fijar una semilla para garantizar reproducibilidad.

Instrucciones

1. Copie y ejecute el código en una celda nueva.

2. Verifique que SciPy sea 1.11 o superior y statsmodels 0.14 o superior. Si no lo son, ejecute `!pip install -U scipy statsmodels` y reinicie la sesión (Entorno de ejecución → Reiniciar sesión).

Código

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
from scipy import stats
import statsmodels
from statsmodels.stats.proportion import (proportions_ztest,
                                           proportion_effectsize)
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.multitest import multipletests

pd.set_option("display.float_format", "{:.4f}".format)
pd.set_option("display.width", 200); pd.set_option("display.max_columns", 30)
SEMILLA = 2026
print("pandas", pd.__version__, "| scipy", scipy.__version__,
      "| statsmodels", statsmodels.__version__)
```

La salida mostrará las versiones instaladas. En Colab es normal que las versiones de pandas sean de la serie 2.x; el código de la guía es compatible con pandas 2.2 y 3.x.

Parte 1. Exploración y estadística descriptiva robusta

Contexto: MercaAndina, empresa peruana de comercio electrónico, necesita describir el comportamiento del ticket de compra y de los tiempos de entrega antes de definir su estrategia logística.

Paso 1.1. Cargar y revisar el dataset

Propósito: verificar dimensiones, tipos de datos y valores faltantes antes de cualquier cálculo.

Instrucciones

1. Cargue el archivo con `pd.read_csv`.
2. Revise el número de filas y columnas, los tipos de datos y los valores faltantes por columna.

Código

```
df = pd.read_csv("pedidos_mercaandina_2026.csv")
print(df.shape)
print(df.dtypes)
print(df.head())
print(df.isna().sum())
```

Resultado esperado

```
(12000, 6)
region      str
canal       str
metodo_pago str
ticket      float64
dias_entrega float64
satisfaccion float64
dtype: object
  region canal  metodo_pago  ticket  dias_entrega  satisfaccion
0  Lima   App      Tarjeta  108.8100         2.0000         5.0000
1  Norte  App      Tarjeta   67.1500         4.0000         5.0000
2  Norte  App      Tarjeta  100.3500         7.0000         3.0000
```

3	Lima	Web	Billetera digital	92.3600	3.0000	4.0000
4	Lima	Web	Tarjeta	18.8700	1.0000	5.0000
region				0		
canal				0		
metodo_pago				0		
ticket				0		
dias_entrega				0		
satisfaccion				180		
dtype:				int64		

Preguntas de control (responda en una celda Markdown)

- ¿Qué variable tiene valores faltantes y cuántos? ¿Qué porcentaje del total representan?
- Clasifique cada variable según su tipo y escala de medición (nominal, ordinal, razón).

Paso 1.2. Resumen robusto del ticket por canal

Propósito: construir una función reutilizable que calcule medidas clásicas y robustas, y aplicarla por grupos.

Instrucciones

1. Defina la función `resumen_robusto` tal como aparece en el código; la usará en varios pasos.
2. Aplíquela al `ticket` agrupando por canal con `groupby(...).apply(...).unstack()`.

Código

```
def resumen_robusto(x):
    x = x.dropna()
    q1, q3 = x.quantile([0.25, 0.75])
    return pd.Series({
        "n": x.count(), "media": x.mean(), "mediana": x.median(),
        "media_rec_10": stats.trim_mean(x, 0.10),
        "desv_est": x.std(), "IQR": q3 - q1,
        "MAD_esc": stats.median_abs_deviation(x, scale="normal"),
        "CV_%": x.std() / x.mean() * 100,
        "asimetria": stats.skew(x), "P90": x.quantile(0.90),
        "P95": x.quantile(0.95)})

tabla_canal = df.groupby("canal")["ticket"].apply(resumen_robusto).unstack()
print(tabla_canal.T.round(2)) # traspuesta: filas = medidas
```

Resultado esperado

canal	App	Web
n	5324.0000	6676.0000
media	163.0800	157.7300
mediana	135.7000	132.1200
media_rec_10	146.2800	141.2400
desv_est	124.4500	127.7100
IQR	104.1800	103.5800
MAD_esc	71.4200	71.2100
CV_%	76.3100	80.9700
asimetria	7.1700	8.8000
P90	279.9600	271.4500
P95	345.5600	330.9000

Preguntas de control (responda en una celda Markdown)

- ¿Por qué la media es mayor que la mediana en ambos canales?
- ¿Qué medida de centro reportaría a la gerencia para describir al «cliente típico»? Justifique con la asimetría.

Paso 1.3. Días de entrega por región

Propósito: describir una variable discreta asimétrica por grupos ordenados geográficamente.

Instrucciones

1. Defina la lista orden con las regiones.
2. Aplique resumen_robusto a dias_entrega por región y muestre solo las columnas indicadas.

Código

```
orden = ["Lima", "Norte", "Sur", "Centro", "Oriente"]
tabla_dias = (df.groupby("region")["dias_entrega"]
              .apply(resumen_robusto).unstack().loc[orden])
cols = ["n", "media", "mediana", "IQR", "CV_%", "asimetria", "P90"]
print(tabla_dias[cols].round(2))
```

Resultado esperado

region	n	media	mediana	IQR	CV_%	asimetria	P90
Lima	5364.0000	1.8500	2.0000	1.0000	49.2100	1.1300	3.0000
Norte	2378.0000	3.1900	3.0000	2.0000	49.9700	1.0400	5.0000
Sur	2054.0000	3.5200	3.0000	3.0000	50.6000	0.9900	6.0000
Centro	1305.0000	3.8300	4.0000	3.0000	49.7600	0.9100	6.0000
Oriente	899.0000	5.2200	5.0000	4.0000	51.4200	0.9000	9.0000

Preguntas de control (responda en una celda Markdown)

- ¿Qué región tiene la mayor mediana de días de entrega y el mayor P90?
- Interprete el P90 de Oriente en términos de la experiencia del cliente.

Paso 1.4. Visualizar las distribuciones

Propósito: confirmar gráficamente lo observado en los resúmenes numéricos.

Instrucciones

1. Ejecute el código para generar el histograma del ticket (recortado en P99) y los diagramas de caja de días de entrega.
2. Observe la posición relativa de la media y la mediana.

Código

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
df["ticket"].clip(upper=df["ticket"].quantile(0.99)).plot.hist(
    bins=50, ax=ax[0], color="gray", edgecolor="black")
ax[0].axvline(df["ticket"].mean(), ls="--", color="black", label="Media")
ax[0].axvline(df["ticket"].median(), ls="-", color="black", label="Mediana")
ax[0].set_title("Ticket (S/) hasta P99"); ax[0].legend()
datos = [df.loc[df.region == r, "dias_entrega"] for r in orden]
ax[1].boxplot(datos)
ax[1].set_xticks(range(1, 6), orden)
ax[1].set_title("Días de entrega por región")
plt.tight_layout(); plt.savefig("fig_paso 1 4.png", dpi=150); plt.show()
```

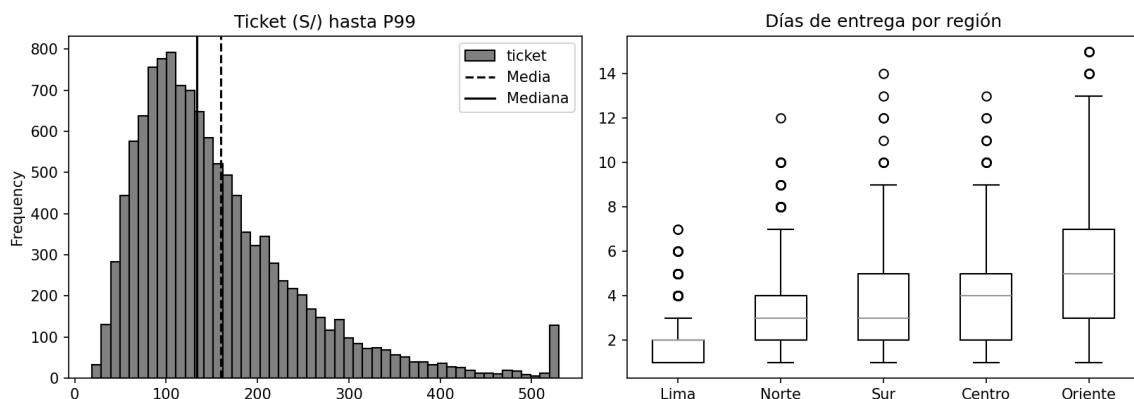


Figura 1. Resultado esperado del paso 1.4

Preguntas de control (responda en una celda Markdown)

- Describa la forma de la distribución del ticket en una oración.
- ¿El diagrama de caja confirma el orden de regiones observado en el paso 1.3?

Parte 2. Detección y tratamiento de valores atípicos

Paso 2.1. Comparar cuatro criterios de detección

Propósito: evidenciar que distintos criterios producen resultados diferentes sobre los mismos datos.

Instrucciones

1. Calcule las marcas de atípico con Tukey (IQR), puntuación z, z robusto (MAD) y Tukey sobre $\log(\text{ticket})$.
2. Guarde cada marca como columna booleana en `df` y cuente cuántos registros marca cada criterio.

Código

```
t = df["ticket"]
q1, q3 = t.quantile([0.25, 0.75]); iqr = q3 - q1
df["out_iqr"] = (t < q1 - 1.5*iqr) | (t > q3 + 1.5*iqr)
df["out_z"] = ((t - t.mean()) / t.std()).abs() > 3
mad = stats.median_abs_deviation(t)
df["out_zrob"] = (0.6745 * (t - t.median()) / mad).abs() > 3.5
lt = np.log(t); lq1, lq3 = lt.quantile([0.25, 0.75]); liqr = lq3 - lq1
df["out_iqr_log"] = (lt < lq1 - 1.5*liqr) | (lt > lq3 + 1.5*liqr)
print(df[["out_iqr", "out_z", "out_zrob", "out_iqr_log"]].sum())
```

Resultado esperado

```
out_iqr      516
out_z        114
out_zrob     372
out_iqr_log  120
dtype: int64
```

Preguntas de control (responda en una celda Markdown)

- ¿Por qué el criterio z detecta menos atípicos que el z robusto? Explique el efecto de enmascaramiento.
- ¿Por qué el criterio IQR aplicado sobre $\log(\text{ticket})$ es más adecuado para esta variable?

Paso 2.2. Impacto de los atípicos en las medidas

Propósito: cuantificar cuánto cambian las medidas clásicas y robustas al excluir los atípicos.

Instrucciones

1. Excluya los registros marcados por `out_iqr_log`.
2. Compare media, mediana y desviación estándar con y sin atípicos, y calcule el cambio porcentual.

Código

```
sin_out = df.loc[~df["out_iqr_log"], "ticket"]
comparacion = pd.DataFrame({
    "con_atipicos": [t.mean(), t.median(), t.std()],
    "sin_atipicos_log": [sin_out.mean(), sin_out.median(), sin_out.std()]},
    index=["media", "mediana", "desv_est"])
comparacion["cambio_%"] = (comparacion.iloc[:, 1]
    / comparacion.iloc[:, 0] - 1) * 100
print(comparacion.round(2))
```

Resultado esperado

	con_atipicos	sin_atipicos_log	cambio_%
media	160.1100	154.4800	-3.5100
mediana	133.6800	133.4000	-0.2100
desv_est	126.3000	87.2900	-30.8900

Preguntas de control (responda en una celda Markdown)

- ¿Qué medida es más sensible a los atípicos? ¿Cuál es más estable?
- ¿Recomendaría eliminar estos registros del análisis de ventas? Proponga una alternativa.

Parte 3. Forma de la distribución e intervalos de confianza

Paso 3.1. Gráficos Q-Q y prueba de normalidad

Propósito: evaluar la normalidad combinando evidencia gráfica y formal.

Instrucciones

1. Extraiga una muestra aleatoria de 500 valores de días_entrega usando la semilla.
2. Genere los gráficos Q-Q de días de entrega y de log(ticket).
3. Aplique Shapiro-Wilk a la muestra y calcule la asimetría de la variable completa.

Código

```
rng = np.random.default_rng(SEMILLA)
muestra = rng.choice(df["dias_entrega"].to_numpy(), 500, replace=False)
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
stats.probplot(muestra, dist="norm", plot=ax[0])
ax[0].set_title("Q-Q días de entrega")
stats.probplot(np.log(df["ticket"].sample(500, random_state=SEMILLA)),
    dist="norm", plot=ax[1]); ax[1].set_title("Q-Q log(ticket)")
plt.tight_layout(); plt.savefig("fig_paso_3_1.png", dpi=150); plt.show()
print("Shapiro días:", stats.shapiro(muestra))
print("Asimetría días:", stats.skew(df["dias_entrega"]))
```

Resultado esperado

Shapiro días: ShapiroResult(statistic=0.8246, pvalue=5.5835e-23)
Asimetría días: 1.6064

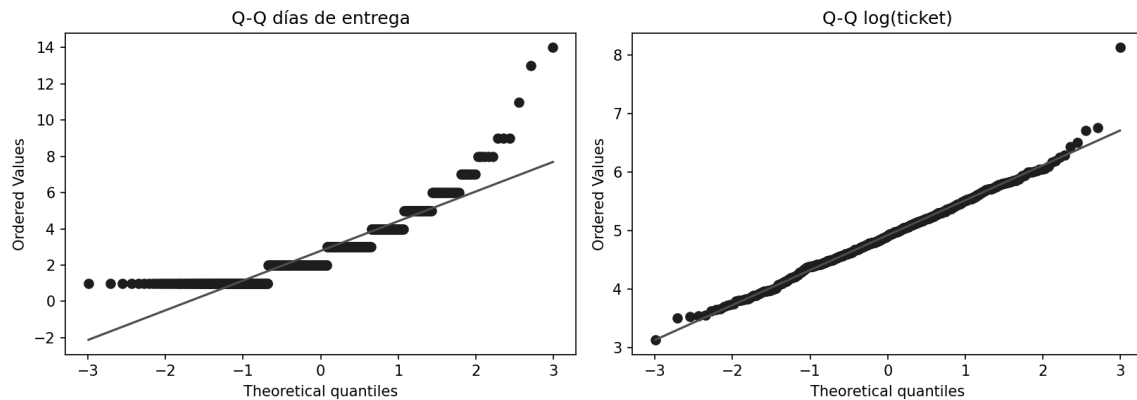


Figura 2. Resultado esperado del paso 3.1

Preguntas de control (responda en una celda Markdown)

- ¿Por qué los puntos del Q-Q de días de entrega forman «escalones»?
- ¿Es razonable aplicar una prueba t a días_entrega con $n = 12\,000$? Fundamente con el Teorema del Límite Central.

Paso 3.2. IC bootstrap del percentil 90 del ticket

Propósito: estimar la incertidumbre de un estadístico sin fórmula analítica sencilla, útil para definir umbrales de envío gratuito.

Instrucciones

1. Defina la función `ic_bootstrap_p90` con 2 000 remuestreos y método percentil.
2. Calcule el P90 y su IC 95% para Lima y Oriente.

Código

```
def ic_bootstrap_p90(x):  
    res = stats.bootstrap((x.to_numpy()),  
                          lambda v, axis: np.percentile(v, 90, axis=axis),  
                          n_resamples=2000, method="percentile",  
                          vectorized=True, random_state=SEMILLA)  
    return np.percentile(x, 90), res.confidence_interval  
  
for r in ["Lima", "Oriente"]:  
    p90, ic = ic_bootstrap_p90(df.loc[df.region == r, "ticket"])  
    print(f"{r:8s} P90 = {p90:8.2f} | IC95% = [{ic.low:.2f} ; {ic.high:.2f}]" )
```

Resultado esperado

```
Lima    P90 =   292.44 | IC95% = [286.81 ; 300.06]  
Oriente P90 =   254.72 | IC95% = [233.97 ; 269.81]
```

Preguntas de control (responda en una celda Markdown)

- ¿Se superponen los intervalos de Lima y Oriente? ¿Qué concluye?
- Si la empresa fija el envío gratuito en el P90 de cada región, ¿qué umbral recomendaría para cada una? Use el límite conservador del IC.

Paso 3.3. IC de la media y efecto del tamaño de muestra

Propósito: comprobar empíricamente que el ancho del IC disminuye con $\sqrt{n}$.

Instrucciones

1. Calcule el IC 95% de la media de días de entrega con la distribución t.
2. Repita el cálculo con submuestras de $n = 100, 400$ y $1\ 600$ y observe el ancho.

Código

```
x = df["dias_entrega"]
ic_t = stats.t.interval(0.95, len(x)-1, loc=x.mean(), scale=stats.sem(x))
print("Media días:", round(x.mean(), 3), "IC95%:", np.round(ic_t, 3))
for n in [100, 400, 1600]:
    s = x.sample(n, random_state=SEMILLA)
    ic = stats.t.interval(0.95, n-1, loc=s.mean(), scale=stats.sem(s))
    print(f"n={n:5d} ancho IC = {ic[1]-ic[0]:.3f}")
```

Resultado esperado

```
Media días: 2.872 IC95%: [2.839 2.905]
n= 100 ancho IC = 0.820
n= 400 ancho IC = 0.380
n= 1600 ancho IC = 0.182
```

Preguntas de control (responda en una celda Markdown)

- ¿En qué proporción se reduce el ancho del IC cada vez que n se cuadruplica?
¿Coincide con la teoría?
- Redacte la interpretación correcta del IC 95% obtenido para la media.

Parte 4. Comparación de grupos

Paso 4.1. Días de entrega: Web vs. App

Propósito: decidir si el canal de compra se asocia con tiempos de entrega distintos.

Instrucciones

1. Plantee por escrito H_0 y H_1 (bilateral, $\alpha = 0.05$) en una celda Markdown antes de ejecutar.
2. Compare medianas y medias, evalúe homogeneidad de varianzas con Levene y aplique Welch y Mann-Whitney.
3. Calcule la correlación biserial de rangos como tamaño del efecto.

Código

```
web = df.loc[df.canal == "Web", "dias_entrega"]
app = df.loc[df.canal == "App", "dias_entrega"]
print("Mediana Web/App:", web.median(), app.median())
print("Media Web/App:", round(web.mean(), 3), round(app.mean(), 3))
print("Levene:", stats.levene(web, app))
print("Welch :", stats.ttest_ind(app, web, equal_var=False))
mw = stats.mannwhitneyu(app, web, alternative="two-sided")
r_rb = 1 - 2 * mw.statistic / (len(app) * len(web))
print("Mann-Whitney:", mw, "| r biserial de rangos:", round(r_rb, 4))
```

Resultado esperado

```
Mediana Web/App: 2.0 2.0
Media Web/App: 2.846 2.905
Levene: LeveneResult(statistic=2.7526, pvalue=0.0971)
Welch : TtestResult(statistic=1.7397, pvalue=0.0819, df=11219.8524)
Mann-Whitney: MannwhitneyuResult(statistic=18012775.0, pvalue=0.1898) | r biserial de rangos: -
0.0136
```

Preguntas de control (responda en una celda Markdown)

- ¿Se rechaza H_0 ? Redacte la conclusión en lenguaje de negocio.

- Compare este resultado con el del ticket en la teoría (donde sí hubo significancia). ¿Por qué un «no significativo» no demuestra que los canales sean iguales?

Paso 4.2. Días de entrega entre regiones: Kruskal-Wallis

Propósito: comparar cinco grupos con una prueba no paramétrica y medir la magnitud del efecto.

Instrucciones

1. Aplique Kruskal-Wallis a los cinco grupos.
2. Calcule épsilon cuadrado: $\epsilon^2 = (H - k + 1) / (N - k)$.
3. Ejecute el ANOVA solo como referencia.

Código

```
grupos = [df.loc[df.region == r, "dias_entrega"] for r in orden]
kw = stats.kruskal(*grupos)
n_total = sum(len(g) for g in grupos); k = len(grupos)
eps2 = (kw.statistic - k + 1) / (n_total - k) # épsilon cuadrado
print("Kruskal-Wallis:", kw, "| epsilon^2 =", round(eps2, 4))
print("ANOVA (referencia):", stats.f_oneway(*grupos))
```

Resultado esperado

```
Kruskal-Wallis: KruskalResult(statistic=3859.8330, pvalue=0.0) | epsilon^2 = 0.3215
ANOVA (referencia): F onewayResult(statistic=1362.8468, pvalue=0.0)
```

Preguntas de control (responda en una celda Markdown)

- ¿Qué proporción de la variabilidad de los días de entrega se asocia a la región? ¿Es un efecto pequeño, mediano o grande?
- ¿Por qué se prefiere Kruskal-Wallis al ANOVA para esta variable?

Paso 4.3. Comparaciones post hoc con corrección múltiple

Propósito: identificar qué pares de regiones difieren controlando el error tipo I.

Instrucciones

1. Genere las 10 combinaciones de pares con itertools.combinations.
2. Aplique Mann-Whitney a cada par y guarde la diferencia de medianas.
3. Corrija los p-valores con Bonferroni, Holm y Benjamini-Hochberg.

Código

```
from itertools import combinations
pares, pvals, difs = [], [], []
for a, b in combinations(orden, 2):
    xa = df.loc[df.region == a, "dias_entrega"]
    xb = df.loc[df.region == b, "dias_entrega"]
    pares.append(f"{a} vs {b}")
    pvals.append(stats.mannwhitneyu(xa, xb).pvalue)
    difs.append(xa.median() - xb.median())
posthoc = pd.DataFrame({"par": pares, "dif_mediana": difs, "p_crudo": pvals})
posthoc["p_bonf"] = multipletests(pvals, method="bonferroni")[1]
posthoc["p_holm"] = multipletests(pvals, method="holm")[1]
posthoc["p_bh"] = multipletests(pvals, method="fdr_bh")[1]
posthoc["signif_holm"] = posthoc["p_holm"] < 0.05
print(posthoc.round(4).to_string(index=False))
```

Resultado esperado

	par	dif_mediana	p_crudo	p_bonf	p_holm	p_bh	signif_holm
	Lima vs Norte	-1.0000	0.0000	0.0000	0.0000	0.0000	True
	Lima vs Sur	-1.0000	0.0000	0.0000	0.0000	0.0000	True
	Lima vs Centro	-2.0000	0.0000	0.0000	0.0000	0.0000	True
	Lima vs Oriente	-3.0000	0.0000	0.0000	0.0000	0.0000	True
	Norte vs Sur	0.0000	0.0000	0.0000	0.0000	0.0000	True
	Norte vs Centro	-1.0000	0.0000	0.0000	0.0000	0.0000	True
	Norte vs Oriente	-2.0000	0.0000	0.0000	0.0000	0.0000	True
	Sur vs Centro	-1.0000	0.0000	0.0000	0.0000	0.0000	True
	Sur vs Oriente	-2.0000	0.0000	0.0000	0.0000	0.0000	True
	Centro vs Oriente	-1.0000	0.0000	0.0000	0.0000	0.0000	True

Preguntas de control (responda en una celda Markdown)

- ¿Cuál es la probabilidad de al menos un falso positivo con 10 pruebas sin corregir ($\alpha = 0.05$)?
- El par Norte vs. Sur tiene diferencia de medianas igual a 0 pero resulta significativo. Explique por qué es posible (recuerde qué compara Mann-Whitney).

Parte 5. Asociación entre variables

Paso 5.1. Canal × método de pago: chi-cuadrado

Propósito: determinar si la preferencia de pago depende del canal de compra.

Instrucciones

1. Construya la tabla de contingencia y la tabla de porcentajes por fila.
2. Aplique `chi2_contingency`, calcule la V de Cramér y verifique que todas las frecuencias esperadas sean ≥ 5 .

Código

```
tabla = pd.crosstab(df["canal"], df["metodo_pago"])
print(tabla)
pct = pd.crosstab(df["canal"], df["metodo_pago"], normalize="index") * 100
print(pct.round(1))
chi = stats.chi2_contingency(tabla)
v = stats.contingency.association(tabla, method="cramer")
print(f"chi2 = {chi.statistic:.3f} | gl = {chi.dof} | "
      f"p = {chi.pvalue:.4f} | V = {v:.4f}")
print("Esperadas >= 5:", (chi.expected_freq >= 5).all())
```

Resultado esperado

```
metodo_pago  Billetera digital  Contra entrega  Tarjeta
canal
App          2181             1046         2097
Web          2803             1276         2597
metodo_pago  Billetera digital  Contra entrega  Tarjeta
canal
App          41.0000          19.6000     39.4000
Web          42.0000          19.1000     38.9000
chi2 = 1.359 | gl = 2 | p = 0.5070 | V = 0.0106
Esperadas >= 5: True
```

Preguntas de control (responda en una celda Markdown)

- ¿Existe asociación entre canal y método de pago? Interprete el valor de V.
- Compare con la asociación región × método de pago de la teoría ($V = 0.18$). ¿Qué variable es más útil para segmentar la estrategia de pagos?

Paso 5.2. Demora vs. satisfacción por región (Spearman)

Propósito: verificar si una asociación global se mantiene dentro de los segmentos (control de la paradoja de Simpson).

Instrucciones

1. Excluya los registros sin satisfacción.
2. Calcule ρ de Spearman entre días de entrega y satisfacción para cada región y para el total.

Código

```
d = df.dropna(subset=["satisfaccion"])
for nombre, g in d.groupby("region"):
    rho = stats.spearmanr(g["dias_entrega"], g["satisfaccion"])
    print(f"nombre:8s n={len(g):5d} rho={rho.statistic:6.3f} "
          f"p={rho.pvalue:.2e}")
print("Global:", stats.spearmanr(d["dias_entrega"], d["satisfaccion"]))
```

Resultado esperado

```
Centro    n= 1282 rho=-0.408 p=1.66e-52
Lima      n= 5279 rho=-0.215 p=5.07e-56
Norte     n= 2343 rho=-0.349 p=3.96e-68
Oriente   n=  889 rho=-0.562 p=3.38e-75
Sur       n= 2027 rho=-0.386 p=8.12e-73
Global: SignificanceResult(statistic=-0.3766, pvalue=0.0)
```

Preguntas de control (responda en una celda Markdown)

- ¿La asociación negativa se mantiene en todas las regiones? ¿En cuál es más fuerte?
- ¿Por qué es importante revisar la correlación por segmentos antes de concluir?

Parte 6. Experimento A/B desde datos crudos

Contexto: el equipo de producto asignó aleatoriamente a 40 250 usuarios, entre el 3 y el 23 de agosto de 2026, a la versión actual del checkout (A) o a una versión simplificada (B). La métrica principal es la tasa de conversión; la métrica secundaria es el monto de compra.

Paso 6.1. Cargar datos y verificar SRM

Propósito: asegurar que la asignación aleatoria funcionó antes de analizar resultados.

Instrucciones

1. Cargue `experimento_checkout_2026.csv` y revise su estructura.
2. Cuente usuarios por variante y aplique una prueba chi-cuadrado de bondad de ajuste contra una división 50/50.

Código

```
ab = pd.read_csv("experimento_checkout_2026.csv", parse_dates=["fecha"])
print(ab.shape); print(ab.head())
conteo = ab["variante"].value_counts().sort_index()
print(conteo)
srm = stats.chisquare(conteo, f_exp=[len(ab)/2, len(ab)/2])
print("Prueba SRM:", srm)
```

Resultado esperado

```
(40250, 7)
  usuario_id      fecha variante dispositivo  gasto_previo_30d  convirtio  monto_compra
0    U100000  2026-08-11      A    Android      46.0600         0         NaN
1    U100001  2026-08-05      A    Desktop      74.7100         1      90.1200
2    U100002  2026-08-09      A    Desktop      73.4100         0         NaN
3    U100003  2026-08-13      B    Android      61.2300         0         NaN
4    U100004  2026-08-17      B    Android      82.1500         0         NaN
variante
A    20078
B    20172
Name: count, dtype: int64
Prueba SRM: Power divergenceResult(statistic=0.2195, pvalue=0.6393)
```

Preguntas de control (responda en una celda Markdown)

- ¿Hay evidencia de Sample Ratio Mismatch? ¿Qué haría si el p-valor fuera menor que 0.001?

Paso 6.2. Prueba de dos proporciones

Propósito: estimar el efecto del nuevo checkout sobre la conversión con su incertidumbre.

Instrucciones

1. Calcule conversiones, usuarios y tasa por variante.
2. Aplique proportions_ztest, el IC 95% de la diferencia, el lift relativo y los IC de Wilson por variante.

Código

```
res = ab.groupby("variante")["convirtio"].agg(conversiones="sum", usuarios="count")
res["tasa_%"] = res.conversiones / res.usuarios * 100
print(res)
z, p = proportions_ztest(res.conversiones[["B", "A"]], res.usuarios[["B", "A"]])
nA, nB = res.loc["A", "usuarios"], res.loc["B", "usuarios"]
pA = res.loc["A", "conversiones"] / nA
pB = res.loc["B", "conversiones"] / nB
se = np.sqrt(pA*(1-pA)/nA + pB*(1-pB)/nB)
dif = pB - pA
print(f"z = {z:.3f} | p = {p:.5f}")
print(f"Diferencia = {100*dif:.2f} pp | IC95% = "
      f"{100*(dif-1.96*se):.2f} ; {100*(dif+1.96*se):.2f} pp")
print(f"Lift relativo = {100*(pB/pA-1):.2f}%")
for v_ in ["A", "B"]:
    ic = proportion_confint(res.loc[v_, "conversiones"],
                           res.loc[v_, "usuarios"], method="wilson")
    print(v_, "IC Wilson:", np.round(ic, 4))
```

Resultado esperado

```
      conversiones  usuarios  tasa_%
variante
A              1917     20078   9.5478
B              2101     20172  10.4154
z = 2.903 | p = 0.0036
Diferencia = 0.87 pp | IC95% = [0.28 ; 1.45] pp
Lift relativo = 9.09%
A IC Wilson: [0.0915 0.0996]
B IC Wilson: [0.1    0.1084]
```

Preguntas de control (responda en una celda Markdown)

- Redacte la conclusión incluyendo diferencia en puntos porcentuales, IC y p-valor.
- Si cada conversión adicional genera S/ 25 de margen y el checkout recibe 300 000 usuarios al mes, estime el impacto mensual con el límite inferior y superior del IC.

Paso 6.3. Métrica secundaria: monto de compra

Propósito: verificar que el nuevo checkout no altera el monto de las compras.

Instrucciones

1. Filtre solo los usuarios que compraron.
2. Compare medianas y aplique Welch y Mann-Whitney entre B y A.

Código

```
compras = ab.dropna(subset=["monto_compra"])
mA = compras.loc[compras.variante == "A", "monto_compra"]
mB = compras.loc[compras.variante == "B", "monto_compra"]
print("Mediana A/B:", round(mA.median(), 2), round(mB.median(), 2))
print("Welch:", stats.ttest_ind(mB, mA, equal_var=False))
print("Mann-Whitney:", stats.mannwhitneyu(mB, mA))
```

Resultado esperado

```
Mediana A/B: 134.47 136.32
Welch: TtestResult(statistic=0.1151, pvalue=0.9083, df=3977.5931)
Mann-Whitney: MannwhitneyuResult(statistic=2032438.5, pvalue=0.6119)
```

Preguntas de control (responda en una celda Markdown)

- ¿El nuevo checkout modifica el monto de compra? ¿Es favorable o desfavorable para el negocio?

Paso 6.4. Análisis por dispositivo con corrección de Holm

Propósito: explorar si el efecto es heterogéneo entre segmentos sin caer en falsos descubrimientos.

Instrucciones

1. Repita la prueba de proporciones para Android, iOS y Desktop.
2. Corrija los tres p-valores con Holm.

Código

```
filas = []
for dsp, g in ab.groupby("dispositivo"):
    c = g.groupby("variante")["convirtio"].agg(["sum", "count"])
    z_, p_ = proportions_ztest(c["sum"][["B", "A"]], c["count"][["B", "A"]])
    tasa = 100 * c["sum"] / c["count"]
    filas.append({"dispositivo": dsp, "tasa_A_%": tasa["A"],
                  "tasa_B_%": tasa["B"], "p_crudo": p_})
seg = pd.DataFrame(filas)
seg["p_holm"] = multipletests(seg["p_crudo"], method="holm")[1]
print(seg.round(4).to_string(index=False))
```

Resultado esperado

dispositivo	tasa A %	tasa B %	p_crudo	p_holm
Android	8.2237	9.1303	0.0199	0.0399
Desktop	11.4390	10.9262	0.3708	0.3708
iOS	10.2477	13.1806	0.0001	0.0003

Preguntas de control (responda en una celda Markdown)

- ¿En qué dispositivos el efecto se mantiene significativo tras la corrección?
- ¿Recomendaría desplegar B solo en algunos dispositivos? ¿Qué riesgo implica decidir con análisis de segmentos no planificados?

Paso 6.5. Potencia y simulación del peeking

Propósito: dimensionar un nuevo experimento y demostrar empíricamente por qué no se debe detener un test al ver $p < 0.05$.

Instrucciones

1. Calcule el tamaño de muestra por grupo para detectar un lift de 5% sobre una base de 9.55% ($\alpha = 0.05$, potencia 80%).
2. Ejecute la simulación: 1 000 experimentos bajo H_0 verdadera, revisando el p-valor cada 1 000 usuarios. La celda puede tardar entre 30 y 60 segundos.

Código

```
es = proportion_effectsize(0.0955 * 1.05, 0.0955)
n_req = NormalIndPower().solve_power(es, alpha=0.05, power=0.80)
print("Usuarios por grupo para lift 5%:", int(np.ceil(n_req)))

rng = np.random.default_rng(SEMILLA)
n_sim, n_max, cada, tasa = 1000, 20000, 1000, 0.0955 # H0 verdadera
falsos_fijo = falsos_peek = 0
for _ in range(n_sim):
    a = rng.random(n_max) < tasa
    b = rng.random(n_max) < tasa
    detenido = False
    for n in range(cada, n_max + 1, cada):
        _, p_ = proportions_ztest([b[:n].sum(), a[:n].sum()], [n, n])
        if p_ < 0.05:
            detenido = True
            break
    falsos_peek += detenido
    _, p_final = proportions_ztest([b.sum(), a.sum()], [n_max, n_max])
    falsos_fijo += p_final < 0.05
print(f"Error tipo I con n fijo : {falsos_fijo/n_sim:.3f}")
print(f"Error tipo I con peeking: {falsos_peek/n_sim:.3f}")
```

Resultado esperado

```
Usuarios por grupo para lift 5%: 60788
Error tipo I con n fijo : 0.053
Error tipo I con peeking: 0.251
```

Preguntas de control (responda en una celda Markdown)

- ¿Cuánto aumenta el error tipo I real al revisar resultados 20 veces?
- ¿Cuántos usuarios por grupo necesitaría para detectar un lift de 5%? Compárelo con los ~20 000 del experimento actual.

Parte 7. Actividad propuesta individual (40 minutos)

7.1 Caso: PagoYa, billetera digital

PagoYa es una billetera digital peruana (ficticia) que migró parte de su tráfico a una nueva versión de su API (v2). El área de Ingeniería de Confiabilidad (SRE) debe decidir si completa la migración. Para ello necesita evidencia estadística sobre latencia, tasa de errores y satisfacción de los usuarios. El archivo **transacciones_api_billetera_2026.csv** contiene 18 000 transacciones con valores faltantes y atípicos intencionales (Anexo A).

7.2 Asignación individual

Cada estudiante analiza **un único tipo de operación**, determinado por el **último dígito de su código de matrícula**. Esta asignación garantiza que cada trabajo sea distinto. El uso de una operación diferente a la asignada anula la actividad.

Último dígito del código	Tipo de operación asignado
0 o 5	Transferencia
1 o 6	Pago QR
2 o 7	Recarga celular
3 o 8	Pago de servicios
4 o 9	Consulta de saldo

Tabla 3. Asignación de la operación según el código de matrícula

Paso 7.1. Filtrar la operación asignada

Propósito: obtener el subconjunto de datos individual.

Instrucciones

1. Reemplace el valor de CODIGO por su código de matrícula.
2. Ejecute la celda y verifique la operación asignada y el número de registros.

Código

```
CODIGO = "20200123" # <-- reemplace con SU código de matrícula
ops = ["Transferencia", "Pago QR", "Recarga celular",
       "Pago de servicios", "Consulta de saldo"]
mi_operacion = ops[int(CODIGO[-1]) % 5]
api = pd.read_csv("transacciones_api_billetera_2026.csv")
mi = api[api["tipo_operacion"] == mi_operacion].copy()
print("Operación asignada:", mi_operacion, "| registros:", len(mi))
```

7.3 Tareas

Desarrolle las siguientes tareas sobre el DataFrame mi. Reutilice y adapte el código del laboratorio guiado. Cada tarea debe incluir código, resultado e interpretación.

Tarea	Descripción	Tiempo
A1. Calidad y descriptiva	Reporte faltantes por columna y documente cómo los tratará. Calcule resumen_robusto de latencia_ms por version_api, incluyendo P95 (métrica típica de SRE).	6 min
A2. Atípicos	Aplique Tukey sobre log(latencia_ms) e indique cuántas transacciones son atípicas por versión. Decida si las conserva y justifique.	4 min
A3. Latencia v1 vs. v2	Plantee H_0 y H_1 . Aplique la prueba adecuada, reporte p-valor y tamaño del efecto (r biserial de rangos). Calcule el IC bootstrap 95% del P95 para cada versión.	10 min
A4. Errores v1 vs. v2	Compare la tasa de error entre versiones con prueba de dos proporciones, IC de la diferencia y lift. Indique si la muestra tiene potencia suficiente calculando el n necesario para detectar la diferencia observada.	7 min
A5. Datacenter y franja	Compare la latencia entre los tres datacenters (Kruskal-Wallis + ϵ^2 + post hoc con Holm) y calcule ρ de Spearman entre latencia y satisfacción.	8 min
A6. Recomendación	Redacte un resumen ejecutivo de 5 a 8 líneas para el líder de SRE: ¿se completa la migración a v2 para su operación? Incluya al menos tres cifras con su incertidumbre.	5 min

Tabla 4. Tareas de la actividad propuesta

Orientaciones

- La latencia es asimétrica: prefiera mediana, percentiles y pruebas no paramétricas; reporte la media solo como referencia.
- Una diferencia significativa no basta: exprese el efecto en milisegundos y en porcentaje respecto de v1.
- La tasa de error es baja ($\approx 1\%$): una ausencia de significancia puede deberse a falta de potencia. Discútalos en A4.
- Sus resultados serán diferentes a los de sus compañeros porque trabajan con operaciones distintas: la calidad del razonamiento es lo que se evalúa.

7.4 Rúbrica de la actividad propuesta

Criterio	Excelente (4)	Aceptable (2–3)	Insuficiente (0–1)
A1–A2 Calidad, descriptiva y atípicos	Tratamiento documentado; medidas robustas bien elegidas e interpretadas	Cálculos correctos con interpretación parcial	Cálculos incompletos o sin interpretación
A3 Latencia	Prueba adecuada, supuestos, efecto e IC bootstrap interpretados	Prueba correcta sin tamaño del efecto o sin IC	Prueba inadecuada o conclusión errónea
A4 Errores y potencia	Prueba, IC, lift y discusión de potencia coherentes	Prueba correcta sin discusión de potencia	Interpreta «no significativo» como «iguales»
A5 Grupos y asociación	Kruskal, ϵ^2 , Holm y Spearman bien interpretados	Falta la corrección múltiple o el tamaño del efecto	Análisis incompleto
A6 Recomendación ejecutiva	Decisión clara, cifras con incertidumbre y riesgos	Decisión con cifras sin incertidumbre	Sin decisión o sin sustento

Tabla 5. Rúbrica de la actividad propuesta (20 puntos)

8. Entregables y evaluación

1. Notebook **Lab04_Apellido_Codigo.ipynb** con todas las celdas ejecutadas en orden (Entorno de ejecución → Reiniciar y ejecutar todo antes de exportar).
2. Exportación en PDF del mismo notebook (Archivo → Imprimir → Guardar como PDF).
3. Carga de ambos archivos en la tarea «Laboratorio 4» del Aula Virtual dentro del plazo indicado por el docente.

Componente	Peso
Laboratorio guiado (Partes 1 a 6): código ejecutado y preguntas de control respondidas	50%
Actividad propuesta (Parte 7) según la rúbrica	40%
Orden, reproducibilidad y claridad del notebook	10%

Tabla 6. Ponderación del laboratorio

8.1 Lista de verificación antes de entregar

- ☐ El notebook se ejecuta completo sin errores desde una sesión nueva.
- ☐ La primera celda contiene nombre, código y fecha.
- ☐ Todas las preguntas de control están respondidas con cifras.
- ☐ La actividad propuesta usa la operación asignada por su código.
- ☐ Cada prueba reporta p-valor, tamaño del efecto e interpretación de negocio.

☐ Se declara el uso de asistentes de IA, si corresponde.

9. Solución de problemas frecuentes

Problema	Causa probable	Solución
FileNotFoundError al leer el CSV	El archivo no se subió o la sesión de Colab se reinició	Vuelva a subir los archivos (paso 0.2) y verifique con !ls
TypeError en stats.bootstrap	Versión antigua de SciPy	!pip install -U scipy y reinicie la sesión
AttributeError: 'statistic' en chi2_contingency	SciPy < 1.10	Actualice SciPy o use chi[0], chi[1], chi[2], chi[3]
Valores ligeramente distintos a los esperados	No se fijó la semilla o se cambió el orden de las celdas	Ejecute desde el inicio con SEMILLA = 2026
La simulación del paso 6.5 tarda demasiado	Recursos limitados	Reduzca n_sim a 300 e indíquelo en el notebook

Tabla 7. Problemas frecuentes y soluciones

Anexo A. Diccionarios de datos

A.1 pedidos_mercaandina_2026.csv

Variable	Tipo	Descripción
region	Nominal	Lima, Norte, Sur, Centro, Oriente
canal	Nominal	Web, App
metodo_pago	Nominal	Tarjeta, Billetera digital, Contra entrega
ticket	Razón	Monto del pedido en soles; incluye compras corporativas atípicas
dias_entrega	Razón (discreta)	Días entre compra y entrega (1–15)
satisfaccion	Ordinal	Calificación 1–5; 180 valores faltantes

A.2 experimento_checkout_2026.csv

Variable	Tipo	Descripción
usuario_id	Identificador	Código único del usuario
fecha	Fecha	Fecha de ingreso al experimento (03/08/2026 – 23/08/2026)
variante	Nominal	A (checkout actual) o B (checkout simplificado)
dispositivo	Nominal	Android, iOS, Desktop
gasto_previo_30d	Razón	Gasto del usuario en los 30 días previos (S/)
convirtio	Binaria	1 si completó la compra, 0 en caso contrario
monto_compra	Razón	Monto de la compra en soles; vacío si no convirtió

A.3 transacciones_api_billetera_2026.csv

Variable	Tipo	Descripción
transaccion_id	Identificador	Código único de la transacción
tipo_operacion	Nominal	Transferencia, Pago QR, Recarga celular, Pago de servicios, Consulta de saldo
version_api	Nominal	v1 (actual) o v2 (nueva)
datacenter	Nominal	Lima, Arequipa, Trujillo
hora	Discreta	Hora del día (0–23)
franja	Nominal	Pico (8–10 h y 18–21 h) o Normal
latencia_ms	Razón	Tiempo de respuesta en milisegundos; 60 faltantes y picos atípicos
error	Binaria	1 si la transacción falló, 0 si fue exitosa
satisfaccion	Ordinal	Calificación 1–5 posterior a la operación; 250 faltantes